

DevSecOps 4-2-Intro-to-Docker

Task 1 - Introduction

In this room, you'll get your first hands-on experience deploying and interacting with Docker containers. Namely, by the end of the room, you will be familiar with the following:

- The basic syntax to get you started with Docker
- Running and deploying your first container
- Understanding how Docker containers are distributed using images
- Creating your own image using a Dockerfile
- How Dockerfiles are used to build containers, using Docker Compose to orchestrate multiple containers
- Applying the knowledge gained from the room into the practical element at the end.

Task 2 - Basic Docker Syntax

Docker can seem overwhelming at first. However, the commands are pretty intuitive, and with a bit of practice, you'll be a Docker wizard in no time. The syntax for Docker can be categorised into four main groups:

- Running a container
- Managing & Inspecting containers
- Managing Docker images
- Docker daemon stats and information

Managing Docker Images

Docker Pull: Before we can run a Docker container, we will first need an image. In this room, we will use the Nginx image to run a web server within a container. Before downloading the image, let's break down the commands and syntax required to download an image. Images can be downloaded using the `docker pull` command and providing the name of the image. For example, `docker pull nginx`. Docker must know where to get this image (such as from a repository which we'll come onto in a later task). Continuing with our example above, let's download this Nginx image!

```
cmnatic@thm:~$ docker pull nginx
Using default tag: latest
```

```
latest: Pulling from library/nginx
-- omitted for brevity --
Status: Downloaded newer image for nginx:latest
cmnatic@thm:~$
```

By running this command, we are downloading the latest version of the image titled "nginx". Images have these labels called *tags*. These *tags* are used to refer to variations of an image. For example, an image can have the same name but different tags to indicate a different version. I've provided an example of how tags are used within the table below:

Docker Image	Tag	Command Example	Explanation
ubuntu	latest	docker pull ubuntu - IS THE SAME AS - docker pull ubuntu:latest	This command will pull the latest version of the "ubuntu" image. If no tag is specified, Docker will assume you want the "latest" version if no tag is specified. It is worth remembering that you do not always want the "latest". This image is quite literally the "latest" in the sense it will have the most recent changes. This could either fix or break your container.
ubuntu	22.04	docker pull ubuntu:22.04	This command will pull version "22.04 (Jammy)" of the "ubuntu" image.
ubuntu	20.04	docker pull ubuntu:20.04	This command will pull version "20.04 (Focal)" of the "ubuntu" image.
ubuntu	18.04	docker pull ubuntu:18.04	This command will pull version "18.04 (Bionic)" of the "ubuntu" image.

When specifying a tag, you must include a colon `:` between the image name and tag, for example, `ubuntu:22.04` (image:tag).

Docker Image x/y/z: The `docker image` command, with the appropriate option, allows us to manage the images on our local system. To list the available options, we can simply do `docker image` to see what we can do. A terminal showing the various arguments we can provide with "docker image"

```
cmnatic@thm:~$ docker image
```

```
Usage:  docker image COMMAND
```

```
Manage images
```

Commands:

build	Build an image from a Dockerfile
history	Show the history of an image
import	Import the contents from a tarball to create a filesystem image
inspect	Display detailed information on one or more images
load	Load an image from a tar archive or STDIN
ls	List images
prune	Remove unused images
pull	Pull an image or a repository from a registry
push	Push an image or a repository to a registry
rm	Remove one or more images
save	Save one or more images to a tar archive (streamed to STDOUT by default)
tag	Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE

Run '**docker image COMMAND --help**' for **more** information on a command.

cmnatic@thm:~\$

- In this room, we are only going to cover the following options for docker images:
 - pull (we have done this above!)
 - ls (list images)
 - rm (remove an image)
 - build (we will come onto this in the "Building your First Container" task)

Docker Image ls: This command allows us to list all images stored on the local system. We can use this command to verify if an image has been downloaded correctly and to view a little bit more information about it (such as the tag, when the image was created and the size of the image). A terminal listing the Docker images that are stored on the host operating system:

```
cmnatic@thm:~$ docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        22.04     2dc39ba059dc   10 days ago   77.8MB
```

```
nginx          latest      2b7d6430f78d    2 weeks ago    142MB
cmnatic@thm:~$
```

For example, in the terminal above, we can see some information for two images on the system:

Repository	Tag	Image ID	Created	Size
ubuntu	22.04	2dc39ba059dc	10 days ago	77.8MB
nginx	latest	2b7d6430f78d	2 weeks ago	142MB

Docker Image rm: If we want to remove an image from the system, we can use `docker image rm` along with the name (or Image ID). In the following example, I will remove the "ubuntu" image with the tag "22.04". My command will be `docker image rm ubuntu:22.04`. It is important to remember to include the *tag* with the image name. A terminal displaying the untagging of an image

```
cmnatic@thm:~$ docker image rm ubuntu:22.04
Untagged: ubuntu:22.04
Untagged: ubuntu@sha256:20fa2d7bb4de7723f542be5923b06c4d704370f0390e4ae9e1c833c8785644c1
Deleted: sha256:2dc39ba059dcd42ade30aae30147b5692777ba9ff0779a62ad93a74de02e3e1f
Deleted: sha256:7f5cbd8cc787c8d628630756bcc7240e6c96b876c2882e6fc980a8b60cdfa274
cmnatic@thm:~$
```

If we were to run a `docker image ls`, we would see that the image is no longer listed. A terminal confirming that our Docker image has been deleted

```
cmnatic@thm:~$ docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    2b7d6430f78d   2 weeks ago    142MB
cmnatic@thm:~$
```

Answer the questions below

| If we wanted to pull a docker image, what would our command look like?

docker pull

If we wanted to list all images on a device running Docker, what would our command look like?

```
docker image ls
```

Let's say we wanted to pull the image "tryhackme" (no quotations); what would our command look like?

```
docker pull tryhackme
```

Let's say we wanted to pull the image "tryhackme" with the tag "1337" (no quotations). What would our command look like?

```
docker pull tryhackme:1337
```

Task 3 - Running Your First Container

The Docker run command creates running containers from images. This is where commands from the Dockerfile (as well as our own input at runtime) are run. Because of this, it must be some of the first syntaxes you learn. The command works in the following way: `docker run [OPTIONS] IMAGE_NAME [COMMAND] [ARGUMENTS...]` the options enclosed in brackets are not required for a container to run. Docker containers can be run with various options - depending on how we will use the container. This task will explain some of the most common options that you may want to use.

First, Simply Running a Container: Let's recall the syntax required to run a Docker container: `docker run [OPTIONS] IMAGE_NAME [COMMAND] [ARGUMENTS...]`. In this example, I am going to configure the container to run:

- An image named "helloworld"
- "Interactively" by providing the `-it` switch in the [OPTIONS] command. This will allow us to interact with the container directly.
- I am going to spawn a shell within the container by providing `/bin/bash` as the [COMMAND] part. This argument is where you will place what commands you want to run within the container (such as a file, application or shell!)

So, to achieve the above, my command will look like the following: `docker run -it helloworld /bin/bash` A terminal showing a container being launched in 'interactive' mode

```
cmnatic@thm-intro-to-docker:~$ docker run -it helloworld /bin/bash
root@30eff5ed7492:/#
```

We can verify that we have successfully launched a shell because our prompt will change to another user account and hostname. The hostname of a container is the container ID (which can be found by using `docker ps`). For example, in the terminal above, our username and hostname are `root@30eff5ed7492`

Running Containers...Continued: Docker containers can be run with various options. The purpose of the container and the instructions set in a Dockerfile determines what options we need to run the container with. To start, I've put some of the most common options you may need to run your Docker container into the table below.

[OPTION]	Explanation	Relevant Dockerfile Instruction	Example
-d	This argument tells the container to start in "detached" mode. This means that the container will run in the background.	N/A	<code>docker run -d helloworld</code>
-it	This argument has two parts. The "i" means run interactively, and "t" tells Docker to run a shell within the container. We would use this option if we wish to interact with the container directly once it runs.	N/A	<code>docker run -it helloworld</code>
-v	This argument is short for "Volume" and tells Docker to mount a directory or file from the host operating system to a location within the container. The location these files get stored is	VOLUME	<code>docker run -v /host/os/directory:/container/directory helloworld</code>

	defined in the Dockerfile		
-p	This argument tells Docker to bind a port on the host operating system to a port that is being exposed in the container. You would use this instruction if you are running an application or service (such as a web server) in the container and wish to access the application/service by navigating to the IP address.	EXPOSE	<code>docker run -p 80:80 webserver</code>
--rm	This argument tells Docker to remove the container once the container finishes running whatever it has been instructed to do.	N/A	<code>docker run --rm helloworld</code>
--name	This argument lets us give a friendly, memorable name to the container. When a container is run without this option, the name is two random words. We can use this open to name a container after the application the container is running.	N/A	<code>docker run --name helloworld</code>

These are just some arguments we can provide when running a container. Again, most arguments we need to run will be determined by how the container is built. However, arguments such as `--rm` and `--name` will instruct Docker on how to run the container. Other arguments include (but are not limited to!):

- Telling Docker what network adapter the container should use
- What capabilities the container should have access to.
- Storing a value into an environment variable

Listing Running Containers: To list running containers, we can use the `docker ps` command. This command will list containers that are currently running - like so. A terminal showing a list of running containers and their information:

```
cmnatic@thm:~/intro-to-docker$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS          NAMES
a913a8f6e30f   cmnatic/helloworld:latest          "sleep"                 1 months ago
Up 3 days      0.0.0.0:8000->8000/tcp              helloworld
cmnatic@thm:~/intro-to-docker$
```

This command will also show information about the container, including:

- The container's ID
- What command is the container running
- When was the container created
- How long has the container been running
- What ports are mapped
- The name of the container

Tip: To list all containers (even stopped), you can use `docker ps -a`:

A terminal showing a list of ALL containers and their information

```
cmnatic@thm:~/intro-to-docker$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS          NAMES
a913a8f6e30f   cmnatic/helloworld:latest          "sleep"                 1 months ago
Up 3 days      0.0.0.0:8000->8000/tcp              helloworld
```

```
00ba1eed0826  gobuster:cmnatic  "./gobuster dir -u  
rL..."  an hour ago  Exited an hour ago practical_khayyam
```

Answer the questions below

What would our command look like if we wanted to run a container interactively?

```
docker run -it
```

What would our command look like if we wanted to run a container in "detached" mode?

```
docker run -d
```

Let's say we want to run a container that will run and bind a webserver on port 80. What would our command look like?

```
docker run -p 80:80
```

How would we list all running containers?

```
docker ps
```

Now, how would we list all containers (including stopped)?

```
docker ps -a
```

Task 4 - Intro to Dockerfiles

Dockerfiles play an essential role in Docker. Dockerfiles is a formatted text file which essentially serves as an instruction manual for what containers should do and ultimately assembles a Docker image. You use Dockerfiles to contain the commands the container should execute when it is built. To get started with Dockerfiles, we need to know some basic syntax and instructions. Dockerfiles are formatted in the following way:

INSTRUCTION argument

First, let's cover some essential instructions:

Instruction	Description	Example
FROM	This instruction sets a build stage for the container as well as setting the base image (operating system). All Dockerfiles must start with this.	FROM ubuntu

RUN	This instruction will execute commands in the container within a new layer.	RUN whoami
COPY	This instruction copies files from the local system to the working directory in the container (the syntax is similar to the <code>cp</code> command).	COPY /home/cmndatic/myfolder/app/
WORKDIR	This instruction sets the working directory of the container. (similar to using <code>cd</code> on Linux).	WORKDIR / (sets to the root of the filesystem in the container)
CMD	This instruction determines what command is run when the container starts (you would use this to start a service or application).	CMD /bin/sh -c script.sh
EXPOSE	This instruction is used to tell the person who runs the container what port they should publish when running the container.	EXPOSE 80 (tells the person running the container to publish to port 80 i.e. <code>docker run -p 80:80</code>)

Now that we understand the core instructions that make up a Dockerfile, let's see a working example of a Dockerfile. But first, I'll explain what I want the container to do:

1. Use the "Ubuntu" (version 22.04) operating system as the base.
2. Set the working directory to be the root of the container.
3. Create the text file "helloworld.txt".

```
# THIS IS A COMMENT
# Use Ubuntu 22.04 as the base operating system of the container
FROM ubuntu:22.04

# Set the working directory to the root of the container
WORKDIR /

# Create helloworld.txt
RUN touch helloworld.txt
```

Remember, the commands that you can run via the `RUN` instruction will depend on the operating system you use in the `FROM` instruction. (In this example, I have chosen Ubuntu. It's important to remember that the operating systems used in containers are usually very

minimal. I.e., don't expect a command to be there from the start (even commands like *curl*, *ping*, etc., may need to be installed.)

Building Your First Container: Once we have a Dockerfile, we can create an image using the `docker build` command. This command requires a few pieces of information:

1. Whether or not you want to name the image yourself (we will use the `-t` (tag) argument).
2. The name that you are going to give the image.
3. The location of the Dockerfile you wish to build with.

I'll provide the scenario and then explain the relevant command. Let's say we want to build an image - let's fill in the two required pieces of information listed above:

1. We are going to name it ourselves, so we are going to use the `-t` argument.
2. We want to name the image.
3. The Dockerfile is located in our current working directory (`.`).

The Dockerfile we are going to build is the following:

```
# Use Ubuntu 22.04 as the base operating system of the container
FROM ubuntu:22.04

# Set the working directory to the root of the container
WORKDIR /

# Create helloworld.txt
RUN touch helloworld.txt
```

The command would look like so:

```
docker build -t helloworld .
```

(we are using the dot to tell Docker to look in our working directory). If we have filled out the command right, we will see Docker starting to build the image. A terminal showing the building process of the "helloworld" image

```
cmnatic@thm:~$ docker build -t helloworld .
Sending build context to Docker daemon 4.778MB
Step 1/3 : FROM ubuntu:22.04
```

```

22.04: Pulling from library/ubuntu
2b55860d4c66: Pull complete
Digest: sha256:20fa2d7bb4de7723f542be5923b06c4d704370f0390e4ae9e1c8
33c8785644c1
Status: Downloaded newer image for ubuntu:22.04
---> 2dc39ba059dc
Step 2/3 : WORKDIR /
---> Running in 64d497097f8a
Removing intermediate container 64d497097f8a
---> d6bd1253fd4e
Step 3/3 : RUN touch helloworld.txt
---> Running in 54e94c9774be
Removing intermediate container 54e94c9774be
---> 4b11fc80fdd5
Successfully built 4b11fc80fdd5
Successfully tagged helloworld:latest
cmnatic@thm:~$

```

Great! That looks like a success. Let's use `docker image ls` to now see if this image has been built. Using the "docker image ls" command to confirm whether or not our image has successfully built

```

cmnatic@thm:~$ docker image ls

```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
helloworld	latest	4b11fc80fdd5	2 minutes ago	77.8MB
ubuntu	22.04	2dc39ba059dc	10 days ago	77.8MB

```

cmnatic@thm:~$

```

Note: Whatever base operating system you list in the `FROM` instruction in the Dockerfile will also be downloaded. This is why we can see two images:

1. helloworld (our image).
2. ubuntu (the base operating system used in our image).

You will now be able to use this image in a container. Refer to the "Running Your First Container" task to remind you how to start a container.

Levelling up Our Dockerfile: Let's level up our Dockerfile. So far, our container will only create a file - that's not very useful! In the following Dockerfile, I am going to:

1. Use Ubuntu 22.04 as the base operating system for the container.

2. Install the "apache2" web server.
3. Add some networking. As this is a web server, we will need to be able to connect to the container over the network somehow. I will achieve this by using the `EXPOSE` instruction and telling the container to expose port `80`.
4. Tell the container to start the "apache2" service at startup. Containers do not have service managers like `systemd` (this is by design - it is bad practice to run multiple applications in the same container. For example, this container is for the apache2 web server - and the apache2 web server only).

```
# THIS IS A COMMENT
FROM ubuntu:22.04

# Update the APT repository to ensure we get the latest version of
apache2
RUN apt-get update -y

# Install apache2
RUN apt-get install apache2 -y

# Tell the container to expose port 80 to allow us to connect to th
e web server
EXPOSE 80

# Tell the container to run the apache2 service
CMD ["apache2ctl", "-D", "FOREGROUND"]
```

For reference, the command to build this would be `docker build -t webserver .` (assuming the Dockerfile is in the same directory as where you run the command from). Once starting the container with the appropriate options (`docker run -d --name webserver -p 80:80 webserver`), we can navigate to the IP address of our local machine in our browser!



Apache2 Default Page

Ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in** [/usr/share/doc/apache2/README.Debian.gz](#). Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
- Configuration files in the `mods-enabled/`, `conf-enabled/` and `sites-enabled/` directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.
- They are activated by symlinking available configuration files from their respective `*-available/` counterparts. These should be managed by using our helpers `a2enmod`, `a2dismod`, `a2ensite`, `a2dissite`, and `a2enconf`, `a2disconf`. See their respective man pages for detailed information.
- The binary is called `apache2` and is managed using `systemd`, so to start/stop the service use `systemctl start apache2` and `systemctl stop apache2`, and use `systemctl status apache2` and `journalctl -u apache2` to check status. `system` and `apache2ctl` can also be used for service management if desired. **Calling**

The web server works! Currently, Apache2 is serving the default files because we have not added our own to the container.

Optimising Our Dockerfile: There's certainly an art to Docker - and it doesn't stop with Dockerfiles! Firstly, we need to ask ourselves why is it essential to optimise our Dockerfile? Bloated Dockerfiles are hard to read and maintain and often use a lot of

unnecessary storage! For example, you can reduce the size of a docker image (and reduce build time!) using a few ways:

1. Only installing the essential packages. What's nice about containers is that they're practically empty from the get-go - we have complete freedom to decide what we want.
2. Removing cached files (such as APT cache or documentation installed with tools). The code within a container will only be executed once (on build!), so we don't need to store anything for later use.
3. Using minimal base operating systems in our `FROM` instruction. Even though operating systems for containers such as Ubuntu are already pretty slim, consider using an even more stripped-down version (i.e. `ubuntu:22.04-minimal`). Or, for example, using Alpine (which can be as small as 5.59MB!).
4. Minimising the number of layers - I'll explain this further below.

Each instruction (I.E. `FROM`, `RUN`, etc.) is run in its own layer. Layers increase build time! The objective is to have as few layers as possible. For example, try chaining commands from `RUN` together like so:

Before:

```
FROM ubuntu:latest
RUN apt-get update -y
RUN apt-get upgrade -y
RUN apt-get install apache2 -y
RUN apt-get install net-tools -y
```

A terminal showing five layers of a Dockerfile being built

```
cmnatic@thm:~$ docker build -t before .
--omitted for brevity--
Step 2/5 : RUN apt-get update -y
---> Using cache
---> 446962612d20
Step 3/5 : RUN apt-get upgrade -y
---> Running in 8bed81c695f4
--omitted for brevity--
cmnatic@thm:~$
```

After:

```
FROM ubuntu:latest
RUN apt-get update -y && apt-get upgrade -y && apt-get install apache2 -y && apt-get install net-tools
```

A terminal showing two layers of a Dockerfile being built

```
cmnatic@thm:~$ docker build -t after .
Sending build context to Docker daemon 4.78MB
Step 1/2 : FROM ubuntu
--> 2dc39ba059dc
Step 2/2 : RUN apt-get update -y && apt-get upgrade -y && apt-get install apache2 -y && apt-get install net-tools
--> Running in a4d4943bcf04
--omitted for brevity--
cmnatic@thm:~$
```

Layer 1 **FROM** ubuntu:latest

Layer 2 **RUN** apt-get update -y && apt-get upgrade -y && apt-get install apache2 -y && apt-get install net-tools

AFTER

Note here how there are now only two build steps (this will be two layers, making the build much quicker). This is just a tiny example of a Dockerfile, so the build time will not be so drastic, but in much larger Dockerfiles - reducing the number of layers will have a fantastic performance increase during the build.

Answer the questions below

What instruction would we use to specify what base image the container should be using?

FROM

What instruction would we use to tell the container to run a command?

RUN

What docker command would we use to build an image using a Dockerfile?

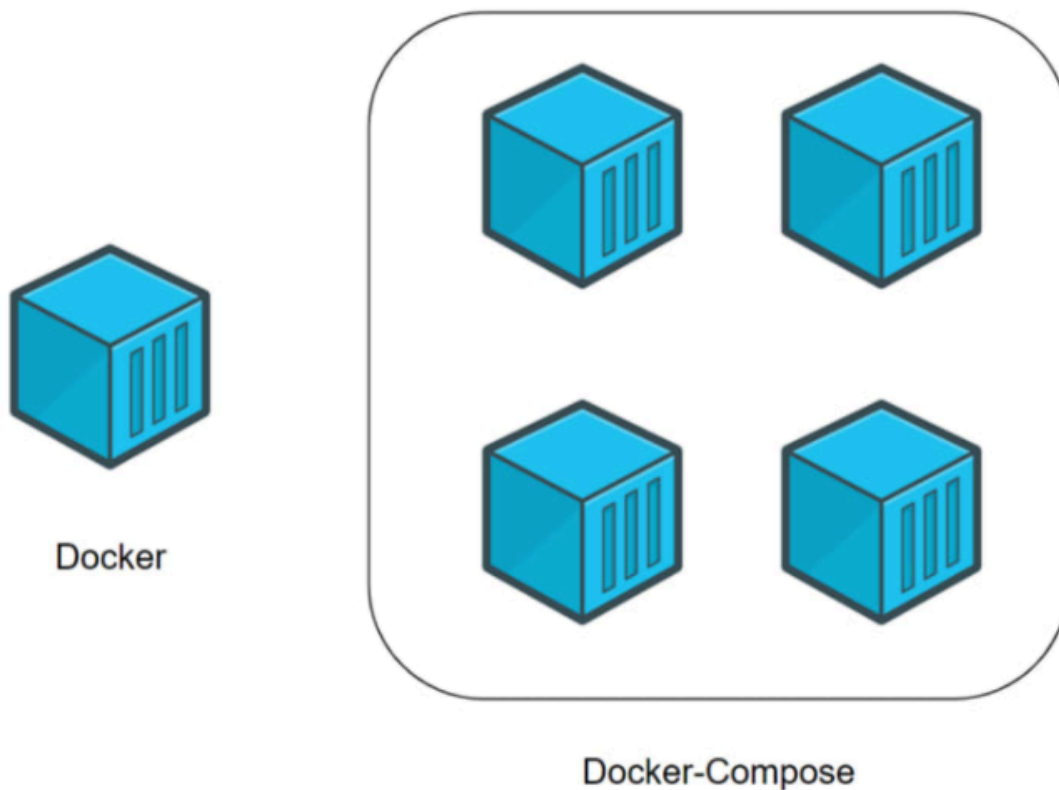
build

| Let's say we want to name this image; what argument would we use?

t

Task 5 - Intro to Docker Compose

Let's first understand what Docker Compose is and why it's worth understanding. So far, we've only interacted with containers individually. Docker Compose, in summary, allows multiple containers (or applications) to interact with each other when needed while running in isolation from one another. You may have noticed a problem with Docker so far. More often than not, applications require additional services to run, which we cannot do in a single container. For example, modern - dynamic - websites use services such as databases and a web server. For the sake of this task, we will consider each application as a "microservice". While we can spin up multiple containers or "microservices" individually and connect them, doing so one by one is cumbersome and inefficient. Docker Compose allows us to create these "microservices" as one singular "service". This illustration shows how containers are deployed together using Docker Compose Vs. Docker:



Before we demonstrate Docker Compose, let's cover the fundamentals of using Docker Compose.

1. We need Docker Compose installed (it does not come with Docker by default). Installing it is out of scope for this room, as it changes depending on your operating system and other factors. You can check out the installation documentation [here](#)(opens in new tab).
2. We need a valid `docker-compose.yml` file - we will come onto this shortly.
3. A fundamental understanding of using Docker Compose to build and manage containers.

I have put some of the essential Docker Compose commands into the table below:

Command	Explanation	Example
up	This command will (re)create/build and start the containers specified in the compose file.	<code>docker-compose up</code>
start	This command will start (but requires the containers already being built) the containers specified in the compose file.	<code>docker-compose start</code>
down	This command will stop and delete the containers specified in the compose file.	<code>docker-compose down</code>
stop	This command will stop (not delete) the containers specified in the compose file.	<code>docker-compose stop</code>
build	This command will build (but will not start) the containers specified in the compose file.	<code>docker-compose build</code>

Note: These are just a few of the possible commands. Check out the [compose documentation](#)(opens in new tab) for all possible options.

A Showcase of Docker Compose: With that said, let's look into how we can use Docker Compose ourselves. In this scenario, I am going to assume the following requirements:

1. An E-commerce website running on Apache
2. This E-commerce website stores customer information in a MySQL database

Now, we could manually run the two containers via the following:

1. Creating the network between the two containers: `docker network create ecommerce`
2. Running the Apache2 webserver container: `docker run -p 80:80 --name webserver --net ecommerce webserver`
3. Running the MySQL Database server: `docker run --name database --net ecommerce webserver`

Container #1

```
docker run --name webservers -p 80:80
```



Container #2

```
docker run --name database
```



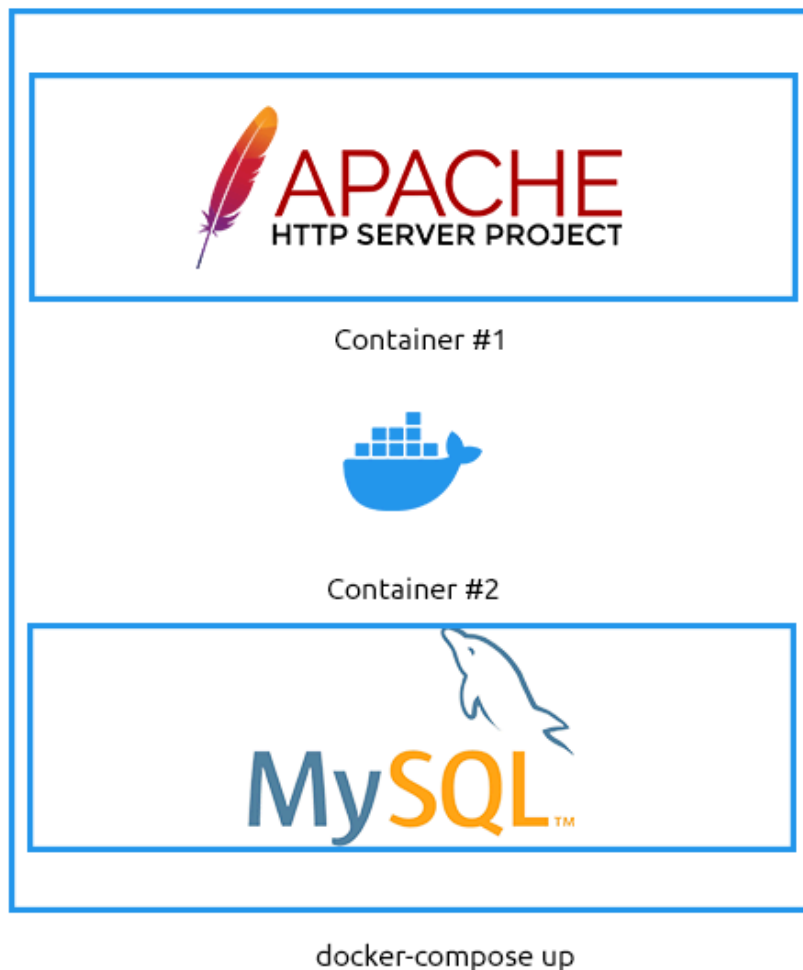
*An illustration shows two containers running independently of each other and is **unable** to communicate with one another.*

...but do we want to do this every time? Or what if we decide to scale up and get many web servers involved? Do we want to do this for every container, every time? I certainly don't.

Instead, we can use Docker Compose via `docker-compose up` to run these containers together, giving us the advantages of:

1. One simple command to run them both
2. These two containers are networked together, so we don't need to go about configuring the network.

3. Extremely portable. We can share our *docker-compose.yml* file with someone else, and they can get the setup working precisely the same without understanding how the containers work individually.
4. Easy to maintain and change. We don't have to worry about specific containers using (perhaps outdated) images.



*An illustration showing two containers deployed as a combined service. These two containers **can** communicate with one another.*

Docker-compose.yml files 101: One file to rule them all. The formatting of a *docker-compose.yml* file is different to that of a Dockerfile. It is important to note that YAML requires indentation (a good practice is two spaces which must be consistent!). First, I'll show some of the new instructions that you will need to learn to be able to write a *docker-compose.yml* file before we go into creating a *docker-compose.yml* file:

Instruction	Explanation	Example
version	This is placed at the top of the file and is used to identify what version of Compose the docker-compose.yml is written for.	'3.3'
services	This instruction marks the beginning of the containers to be managed.	services:
name (replace value)	This instruction is where you define the container and its configuration. "name" needs to be replaced with the actual name of the container you want to define, i.e. "webserver" or "database".	webserver
build	This instruction defines the directory containing the Dockerfile for this container/service. (you will need to use this or an image).	./webserver
ports	This instruction publishes ports to the exposed ports (this depends on the image/Dockerfile).	'80:80'
volumes	This instruction lists the directories that should be mounted into the container from the host operating system.	'./home/cmnnatic/webserver/:/var/www/html'
environment	This instruction is used to pass environment variables (not secure), i.e. passwords, usernames, timezone configurations, etc.	MYSQL_ROOT_PASSWORD=helloworld
image	This instruction defines what image the container should be built with (you will need to use this or build).	mysql:latest
networks	This instruction defines what networks the containers will be a part of. Containers can be part of multiple networks (i.e. a web server can only contact one database, but the database can contact multiple web servers).	ecommerce

Note: These are just some of the possible instructions possible. Check out the [compose file\(opens in new tab\)](#) documentation for all possible instructions.

With that said, let's look at our first docker-compose.yml file. This *docker-compose.yml* file assumes the following:

1. We will run one web server (named web) from the previously mentioned scenario.
2. We will run a database server (named database) from the previously mentioned scenario.
3. The web server is going to be built using its Dockerfile, but we are going to use an already-built image for the database server (MySQL)
4. The containers will be networked to communicate with each other (the network is called ecommerce).
5. Our directory listing looks like the following:
6. docker-compose.yml
7. web/Dockerfile

Here is what our docker-compose.yml file would look like (as a reminder, it is essential to pay attention to the indentation):

```
version: '3.3'
services:
  web:
    build: ./web
    networks:
      - ecommerce
    ports:
      - '80:80'

  database:
    image: mysql:latest
    networks:
      - ecommerce
    environment:
      - MYSQL_DATABASE=ecommerce
      - MYSQL_USERNAME=root
      - MYSQL_ROOT_PASSWORD=helloworld
```

```
networks:
  ecommerce:
```

Answer the questions below

I want to use docker-compose to start up a series of containers. What argument allows me to do this?

up

I want to use docker-compose to delete the series of containers. What argument allows me to do this?

down

What is the name of the .yml file that docker-compose uses? Note: for this question, you will need to include the .yml file extension in your answer

docker-compose.yml

Task 6 - Intro to the Docker Socket

This task will explain how Docker interacts between the operating system and the container. When you install Docker, there are two programs that get installed:

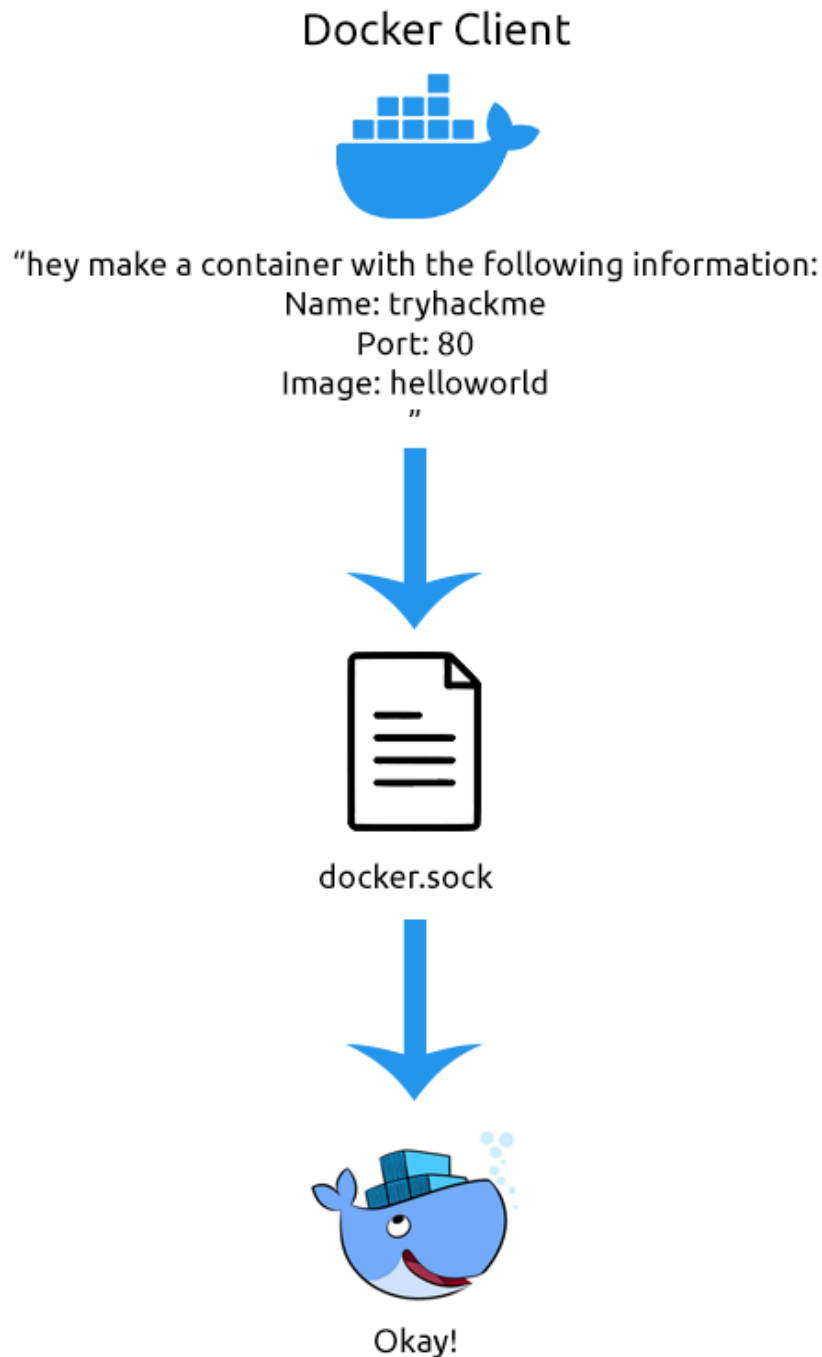
1. The Docker Client
2. The Docker Server

Docker works in a client/server model. Specifically, these two programs communicate with each other to form the Docker that we know and love. Docker achieves this communication using something called a socket. Sockets are an essential feature of the operating system that allows data to be communicated. For example, when using a chat program, there could be two sockets:

1. A socket for storing a message that you are sending
2. A socket for storing a message that someone is sending you.

The program will interact with these two sockets to store or retrieve the data within them! A socket can either be a network connection or what is represented as a file. What's important to know about sockets is that they allow for Interprocess Communication (IPC). This simply means that processes on an operating system can communicate with each other!

In the context of Docker, the Docker Server is effectively just an API. The Docker Server uses this API to **listen** for requests, whereas the Docker Client uses the API to **send** requests. For example, let's take this command: `docker run helloworld`. The Docker Client will request the Docker server to run a container using the image "helloworld". Now, whilst this explanation is fairly basic, it is the essential premise of how Docker works. Let's look at the following diagram to show this process in action:



What's interesting is that because of this, we can interact with the Docker Server using commands like `curl` or an API developer tool such as Postman. Now, using this is out of the scope for this room, but I'll demonstrate communicating with the Docker Server using Postman to list all images that are stored on the operating system:

Finally, it's important to note that because of this, the host machine running Docker can be configured to process commands sent from another device. This is an extremely dangerous vulnerability if it is not correctly configured because it means someone can remotely stop, start, and access Docker containers. Despite this, there are use cases where this feature of Docker is extremely helpful! We will cover this in further detail in a later room!

Answer the questions below

| What does the term "IPC" stand for?

Interprocess Communication

| What technology can the Docker Server be equalled to?

API

Task 7 - Practical

Deploy the lab machine attached to this task by pressing the green "Start Lab Machine" button. After fully loading, the lab machine will appear in split view in your web browser. If you don't see the VM, click the blue "Show Split View" button located at the top right near the top of this page.

Answer the questions below

| Connect to the machine. What is the name of the container that is currently running?

CloudIsland

Use Docker to start a web server with the "webserver" image (no quotations). You will need to run the container with port 80. After starting the container, try to connect to `https://LAB_WEB_URL.p.thmlabs.com/` (opens in new tab) in your browser.

| What is the flag?

THM{WEBSERVER_CONTAINER}

Run the container get the flag

10-113-178-217.reverse-pro...
10-113-178-217.reverse-proxy.cell-prod-eu-cent...
Контакты Imported From... TryHackMe Post...
Congrats! Have a flag: THM{WEBSERVER_CONTAINER}

Check your Internet connection or proxy settings

Last login: Wed Sep 16 12:29:27 2026 from 10.113.89.41
cmnatic@thm-intro-to-docker:~\$ docker image ls
REPOSITORY TAG IMAGE ID CREATED SIZE
webserver latest 1a2769751f0a 3 years ago 223MB
<none> <none> 5bc250eb793f 3 years ago 223MB
<none> <none> 5af755feff52 3 years ago 223MB
<none> <none> 68a438a5ade0 3 years ago 223MB
cloudisland latest eae73f61ae73 3 years ago 77.8MB
ubuntu 22.04 2dc39ba059dc 4 years ago 77.8MB
ubuntu latest 2dc39ba059dc 4 years ago 77.8MB
cmnatic@thm-intro-to-docker:~\$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
0f24bed8b5ef cloudisland "tail -f /dev/null" 3 years ago Up 2 minutes CloudIsland
cmnatic@thm-intro-to-docker:~\$ docker run -d -p
flag needs an argument: 'p' in -p
See 'docker run --help'.
cmnatic@thm-intro-to-docker:~\$ docker run -d -p 80:80 webserver
16411e21aea5dfee2f96e2b8e8049014f5dc1f89b554f40ac8d4f0da2dbc0f25
cmnatic@thm-intro-to-docker:~\$ dockerps
dockerps: command not found
cmnatic@thm-intro-to-docker:~\$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
16411e21aea5 webserver "apache2ctl -D FOREG..." 14 seconds ago Up 12 seconds 0.0.0.0:80->80/tcp, :::80->80/tcp
kind newton
0f24bed8b5ef cloudisland "tail -f /dev/null" 3 years ago Up 3 minutes
CloudIsland
cmnatic@thm-intro-to-docker:~\$